

Module 3: Linked List

- 1.1. Introduction, Representation of Linked List, Linked List vs Arrays.
- 1.2. Types of Linked list - Singly Linked List, Circular Linked list, Doubly Linked list, Operations on Singly and Doubly linked list
- 1.3. Stack and Queue using Linked List.
- 1.4. Singly Linked List, Application - Polynomial Representation and Addition.

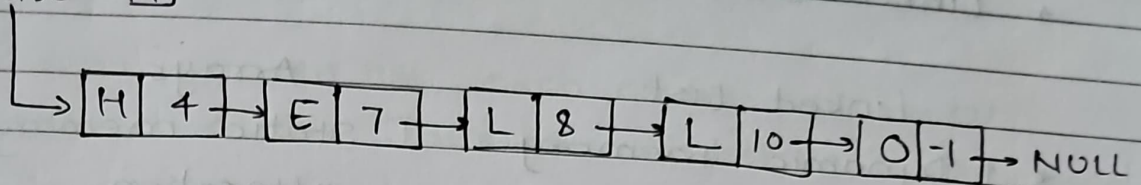
* Introduction.

Linked list is a collection of data elements stored in such a manner such that each element points to next element in the list. Data elements are also called as nodes. Each node has two parts - ^{Info/}Data and ^{Info/}Next. ^{Info/}Data part contains the data element and Next part contains the address of the next node.

Representation of Linked Lists

Unlike arrays, Linked lists do not occupy contiguous locations in memory. Instead they are stored at discrete locations. The beginning of the list is denoted by a special pointer called START and end of the list is denoted by a pointer called NULL.

Start → [1]



Data pointer.

0		
Start → 1	H	4
2		
3		
4	E	7
5		
6		
7	L	8
8	L	10
9		
10	O	-1 → NULL / end.

* Implementation of linked list.

1. Declaring list node.
2. Defining linked list operation.

struct node

{

int INFO;

~~struct~~ struct node * NEXT;

};

* Linked Lists vs Arrays.

Linked List

Arrays.

* 1. Dynamic memory allocation

1. Static memory allocation.

2. Size is not fixed
it can be changed during runtime

2. Size is fixed.

3. Elements are stored at discrete locations

3. Elements are stored at contiguous memory locations.

4. Can increase / decrease in size resulting in efficient memory utilization

4. Remains fixed in size resulting in wastage of memory.

5. Insertion and deletion in the middle is easier

5. Insertion and deletion of elements requires shuffling.

* Types of Linked List

1. Singly Linked list

2. Doubly Linked list

3. Circular Linked list.

1. Singly Linked List.

In singly linked list, each node points at successive nodes. Thus, the list can be traversed in the forward direction only.

Operations on singly linked list.

1. Insert

2. Delete

3. Traverse

4. Search

5. Count No. of Nodes,

Algorithm for traversing

Step 1: Start

Step 2: set $\text{ptr} = \text{start}$

Step 3: while $\text{ptr} \neq \text{NULL}$

$\text{ptr} \rightarrow \text{data}$

$\text{ptr} = \text{ptr} \rightarrow \text{next}$

[end of loop]

Step 4: Exit.

Algorithm for counting number of nodes.

Step 1: start

Step 2: Set $\text{ptr} = \text{start}$

Step 3: while $\text{ptr} \neq \text{NULL}$

$\text{count} = \text{count} + 1$

$\text{ptr} = \text{ptr} \rightarrow \text{next}$

[end of loop]

Step 4: Print count

Step 5: Exit

PAGE No.
 DATE / /

Algorithm for searching a node in linked list.

Step 1: Start

Step 2: set $ptr = start$

Step 3: while $ptr \neq NULL$

if $val = ptr \rightarrow data$

set $pos = ptr$

go to step 5.

else

set $ptr = ptr \rightarrow next$

[end of if]

[end of loop]

Step 4: ~~set~~ Print pos

Step 5: Exit

Algorithm for insertion.

1 At the beginning

Step 1: Start

Step 2: if $AVAIL = NULL$

print "OVERFLOW"

goto step 3

else

set ~~AVAIL~~ $newnode = AVAIL$

$AVAIL = AVAIL \rightarrow next$

$newnode \rightarrow data = value$

$newnode \rightarrow next = start \rightarrow next$

$start = newnode$

[end of if]

Step 3: exit.

2. At the end

Step 1: Start

Step 2: IF $AVAIL = NULL$

print OVERFLOW

goto step. 3.

~~Step~~ else

newnode = AVAIL

$AVAIL = AVAIL \rightarrow NEXT$

newnode $\rightarrow$ data = value

newnode $\rightarrow$ next = NULL.

ptr = start.

while $ptr \rightarrow next \neq NULL$

ptr = ptr $\rightarrow$ next

~~ptr = next~~ [end of loop]

Set $ptr \rightarrow next = newnode$ [end of if]

Step 3: exit

3. In the middle

Step 1: Start

Step 2: IF $AVAIL = NULL$

print OVERFLOW and goto step. 3

else

newnode = avail.

avail = avail $\rightarrow$ next.

newnode $\rightarrow$ data = val

~~new~~ ptr = start

preptr = ptr.

while $ptr \rightarrow Data \neq num$.

preptr = ptr

ptr = ptr $\rightarrow$ next [end of loop]

preptr $\rightarrow$ next = newnode
 newnode $\rightarrow$ next = ptr. [end of if]

Step 3: Exit

Algorithm for deletion

1. Deletion from

1. At the beginning

Step 1: Start

Step 2: if start = NULL

print UNDERFLOW and goto step 3

else

ptr = start

start = start $\rightarrow$ next

free ptr. [end of if]

Step 3: Exit.

2 At the end

Step 1: Start

Step 2: IF start = NULL

print "UNDERFLOW" and goto step 3

else

Set ptr = start

while (ptr $\rightarrow$ next $\neq$ NULL)

~~ptr $\rightarrow$ next = NULL~~

preptr = ptr

ptr = ptr $\rightarrow$ next.

[end of loop]

Set preptr $\rightarrow$ next = null

free ptr. [end of if]

Step 3: Exit

3. in the middle

Step 1: Start

Step 2: IF Start = NULL

print UNDERFLOW and goto step 3
else

ptr = start

while (ptr → data) = num

preptr = ptr

ptr = ptr → next

[end of loop]

preptr → next = ptr → next

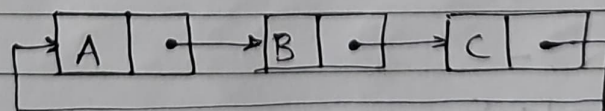
[end of while] free ptr

[end of if]

Step 3: Exit

2 Circular Linked List

The only difference between singly linked list and circular linked list is that the last node of circular linked list is connected to its first node i.e. pointer of the last node points to first node.



Logical Representation

The main advantage of circular linked list is that it allows traversal of the complete list from any of its nodes.

Operations on circular linked list -
Insert, Search, Delete

data ptr

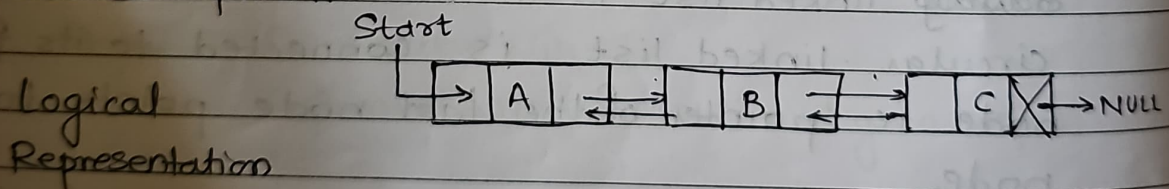
1		
2	A	4
3		
4	B	5
5	C	2

Memory Representation

3. Doubly linked list.

Each node of doubly linked list has three parts - INFO, NEXT and previous. In doubly linked list, the linked pair list points to its next node using next pointer and to its previous node using previous pointer.

The NEXT part of last node contains a NULL value and the beginning of the list is indicated with the help of a special pointer called FIRST.



Declaration :-

```

struct node {
    int INFO;
    struct node *NEXT;
    struct node *PREVIOUS;
};
  
```

Memory Representation

data previous next

0			
Start → 1	A	-1	4
2			
3			
4	B	1	6
5			
6	C	6	-1 → NULL
7			
8			
9			

The main advantage of doubly linked list is that it allows both forward and backward traversal

Operations on doubly linked list are as follows:-

1. Insert
2. Delete
3. Search

1. Insertion at the Beginning.

Algorithm:

Step 1: Start

Step 2: if $avail = NULL$

print "OVERFLOW" and exit.

else

set $newnode = avail$

set $avail = avail \rightarrow next$

set $newnode \rightarrow data = value$

set $newnode \rightarrow prev = NULL$

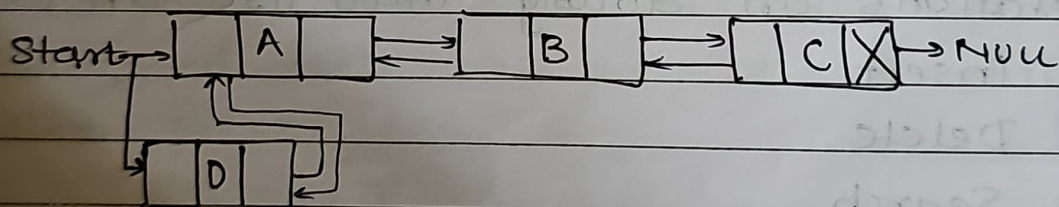
set $newnode \rightarrow next = start \rightarrow next$

set $start \rightarrow prev = newnode$

set $start = newnode$. [end of if]

Step 3: exit

Representation.



2 Insertion at the Middle

Algorithm.

Step 1: Start

Step 2: if avail = NULL
print "OVERFLOW" and exit
else

set newnode = avail

set avail = avail → next

set newnode → data = Value

set ptr = start

Repeat step 3 while ptr → data ≠ value

set newnode → next = ptr → next

set newnode → prev = ptr → prev

set ptr → prev = newnode

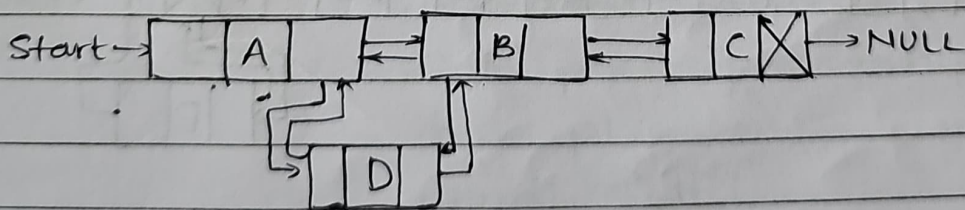
set ptr → prev → next = newnode
[End of IF]

Step 3: ptr → ptr → next

[End of loop]

Step 4: exit

Representation



PAGE No.
 DATE / /

3. Inserting a node at the end

Algorithm

Step 1: Start

Step 2: If avail = NULL

print "OVERFLOW" and exit

else

set avail = avail → next

set newnode = avail

set newnode → data = value

set newnode → next = NULL

set ptr = start

Repeat step 3 while ptr → next ≠ NULL

set newnode → prev = ptr

set ptr → next = newnode

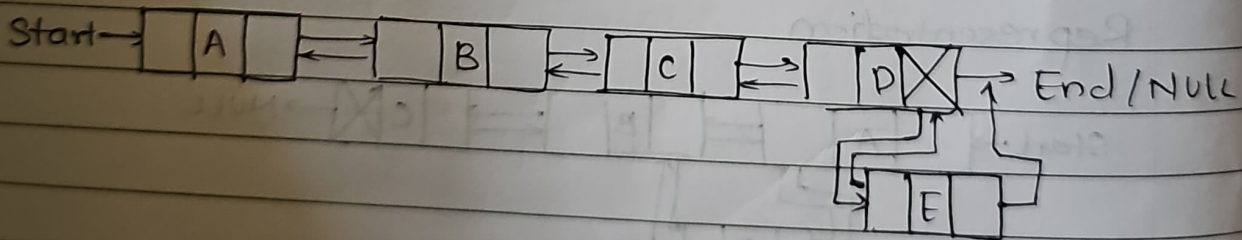
[end of if]

Step 3: Set ptr = ptr → next

[end of loop]

Step 4: Exit

Representation



4. Deleting a node in the beginning.

Algorithm:

Step 1: Start

Step 2: IF Start = NULL

print "UNDERFLOW" and exit.

else

set ptr = start

Set start = start → next

set start → prev = NULL

free ptr

[end of if]

Step 3: Exit

5. Deleting a node at the end

Algorithm:

Step 1: Start

Step 2: IF Start = NULL

print "UNDERFLOW" and exit.

else

set ptr = start

while ptr → next ≠ NULL

repeat step 3.

Set ptr → prev → next = NULL

free ptr

[end of if]

Step 3: ptr = ptr → next. [end of loop]

Step 4: Exit.

PAGE No. / /
DATE / /

6. Deleting a node from the middle

Algorithm:

Step 1: Start.

Step 2: IF $start = NULL$

print UNDERFLOW and exit
else

ptr set $ptr = start$

Repeat step 3 while $ptr \rightarrow data \neq NUM$

Set $ptr \rightarrow prev \rightarrow next = ptr \rightarrow next$

Set $ptr \rightarrow next \rightarrow prev = ptr \rightarrow prev$

free ptr [end of if]

Step 3: $ptr = ptr \rightarrow next$

[end of loop]

Step 4: Exit

* Implementation of stacks and queues using linked list.

1 Implementation of stacks using linked list

Linked implementation of stacks dynamically allocates memory at run time, so the stack takes up the memory required by it and there is no memory wastage.

A. Algorithm for Push operation.

Step 1: Start

Step 2: Assign memory for new node

$\text{new_node} = (\text{struct stack}^*) \text{malloc}$
 $(\text{size of } (\text{struct stack}))$

Step 3: Set new node $\rightarrow$ data = value

Step 4: If top = NULL

Set new node $\rightarrow$ next = NULL

~~else~~

set top = new_node

else

set newnode $\rightarrow$ next = top

set top = new_node

[End of IF]

Step 5: exit

B. Algorithm for Pop operation.

Step 1: Start

Step 2: If top = NULL

print "UNDERFLOW" and exit

else

set ptr = top

set top = top $\rightarrow$ next

free ptr

[End of IF]

Step 3: exit

2. Implementation of queues using linked list.

The linked implementation of queues involves dynamically allocating memory space at run time. This prevents memory wastage as the queue only takes the memory required for holding the data elements.

A. Algorithm for enqueue

Step 1: Start

Step 2: Allocate memory for newnode
`set newnode = (struct queue*) malloc
(size of (struct queue))`

Step 3: Set `newnode → data = value`

Step 4: IF `front = NULL`

`set front = rear = newnode`

`set front → next = rear → next =`

`newnode NULL`

else

`set rear → next = newnode`

`set rear = newnode`

`set rear → next = NULL`

[end of if]

Step 5: exit

B. Algorithm for dequeue.

Step 1: Start

Step 2: IF front = rear = NULL

Print "UNDERFLOW" and exit

Step 3: Setup

else

set ptr = front

set front = front → next

free ptr

[End of IF]

Step 3: exit

* Application of Linked List

1. Implementation of stacks & queues using linked list.
2. Polynomial representation.

eg. $6x^3 + 9x^2 + 7x + 1$

Representation.



Declaration:

```

struct node {
    float coefficient;
    int exponent;
    struct node *ptr;
}
    
```